MAGNUS SIS

Sistema Web de Gestión de Inventario

Modelado lógico · prototipo visual · propuesta de esqueleto Django

INTEGRANTES

- Kevin Julian Sanchez Perez
- Luis Esteban Medina
- Sergio Neira
- Jose Gabriel Cabrera
- Juan Jose Osorio Quitian
- Brayan Steven Sanchez
- Hilver Galindo

Estado: mockups estáticos y esqueleto Django preparados. La lógica CRUD y la conexión MySQL siguen pendientes.

Asignatura: Lenguaje de Programación 1 · 2026-2

Stack: Python · Django MVT · HTML/CSS · SQLite local / MySQL objetivo

Fecha: Octubre de 2026

Contenido y alcance

Entrega inicial: modelado, prototipo visual y esqueleto ejecutable.

CONTENIDO

1. Modelado: flujos, casos de uso, clases y arquitectura.
2. Prototipado: navegación y seis pantallas de referencia.
3. Esqueleto: app Django, modelos, stubs, rutas y configuración.
4. Estado, datos y enlace al repositorio.

ALCANCE FUNCIONAL PROPUESTO

Gestionar productos y categorías: registrar, consultar, buscar, actualizar y eliminar productos; validar datos y representar existencias. Facturación, pagos y reportes complejos quedan fuera.

Estado del código: la app Django, las rutas, modelos y stubs están creados. Todavía no resuelven la lógica CRUD ni persisten productos. SQLite está configurado para desarrollo local.

Mockups: las seis capturas son HTML/CSS estáticos. Sus productos, SKU, cantidades, métricas y estados son ejemplos visuales, no datos guardados.

1. Diagramas lógicos (modelado del sistema)

1.1 Diagrama de flujo - Gestión de Productos

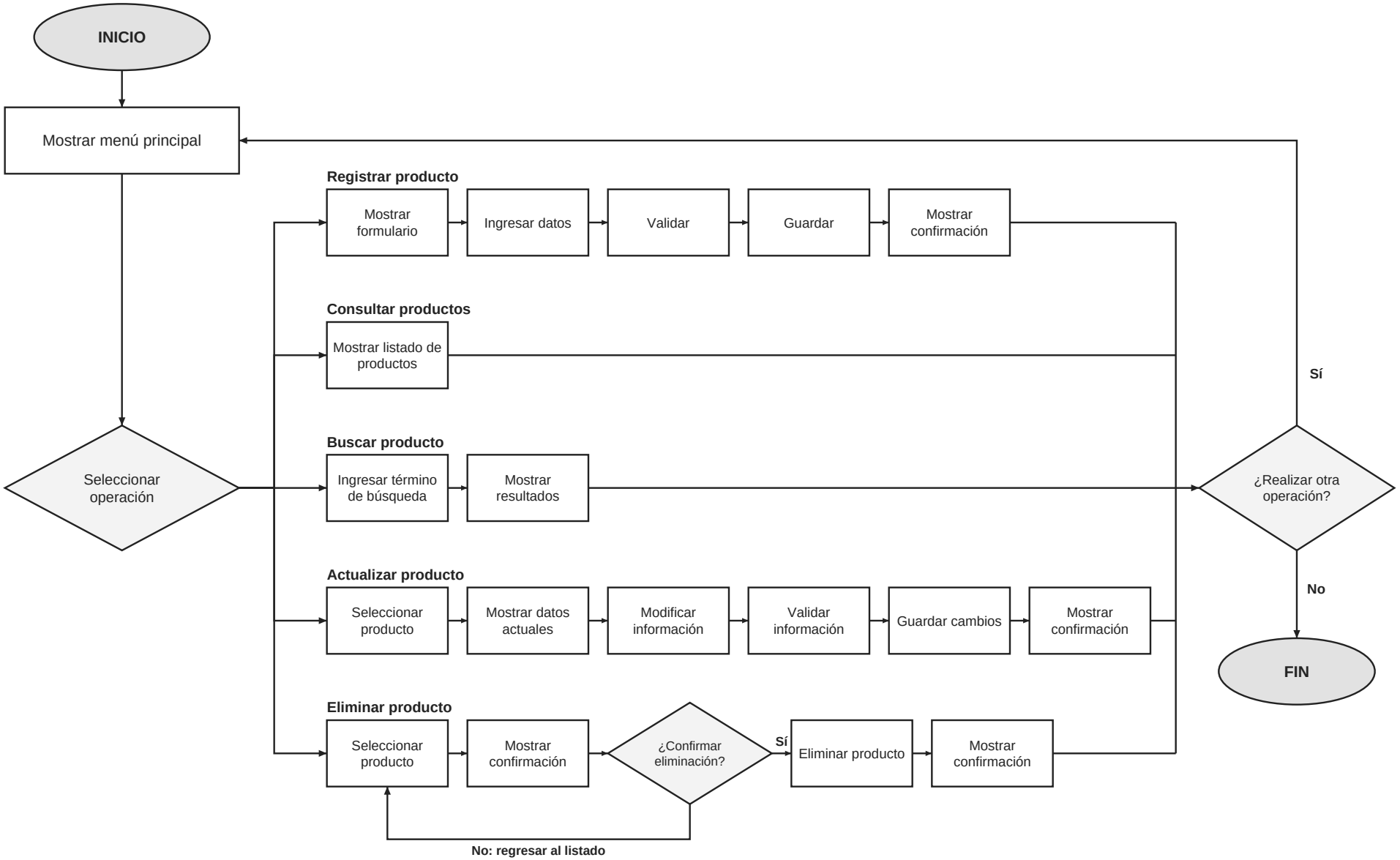


Figura 1. Flujo general: menú, selección de operación y rutas de registrar, consultar, buscar, actualizar y eliminar.

1.2 Diagrama de flujo - Registro de Producto

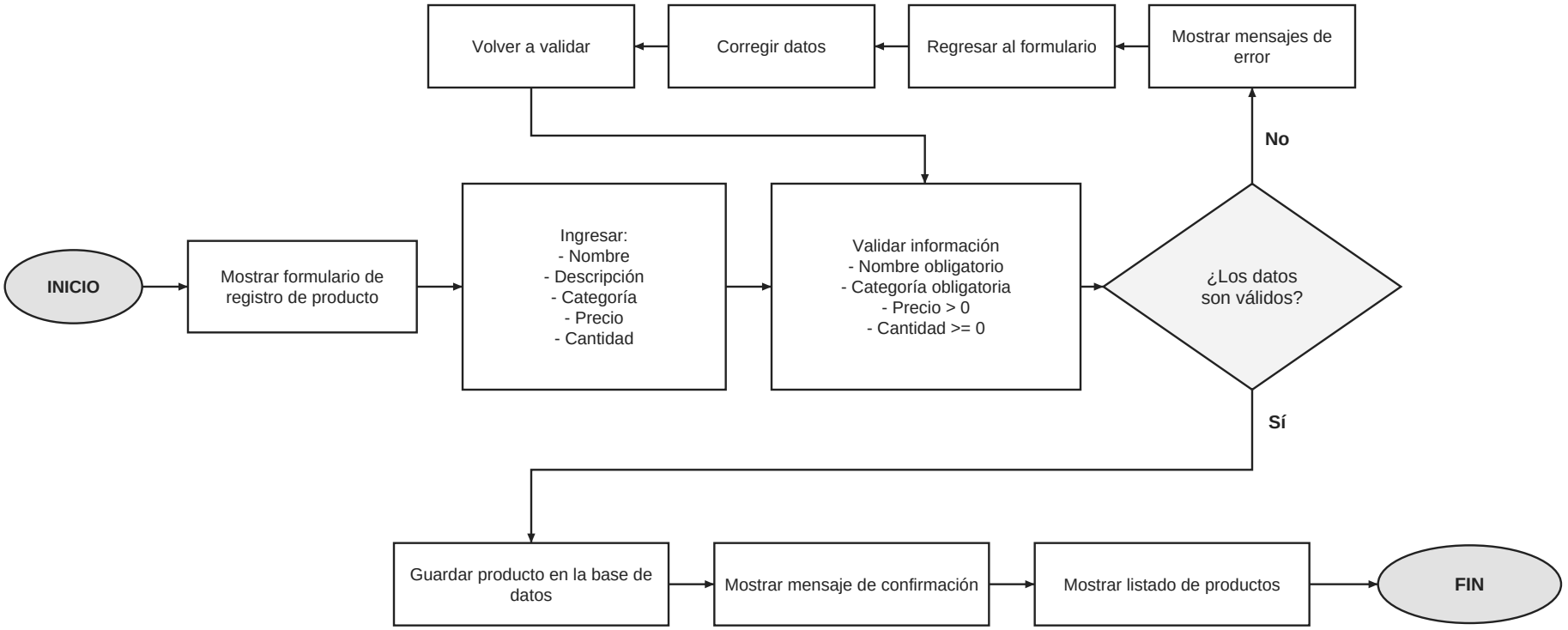
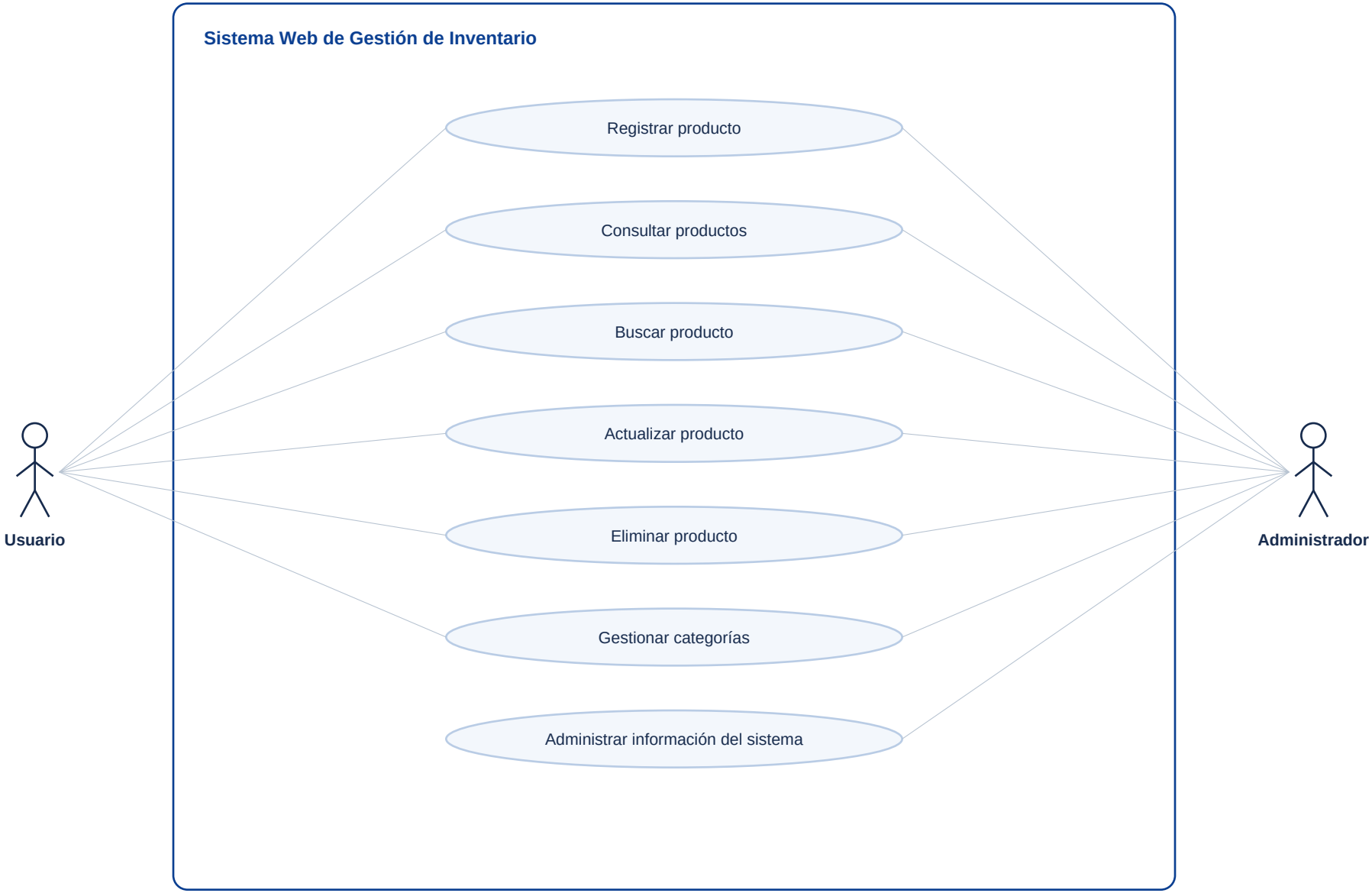


Figura 2. Proceso crítico de validación de entrada con ciclo de corrección. Validaciones: nombre y categoría obligatorios, precio mayor que cero, cantidad mayor o igual a cero.

1.3 Casos de uso

El Administrador puede realizar las operaciones del Usuario y acceder al panel de Django.



Asociaciones simples. Los perfiles y permisos son conceptuales; el mockup muestra un usuario administrador de ejemplo y no implementa autenticación.

Detalle de actores y acciones

Actor	Acciones que puede ejecutar
Usuario	Registrar, consultar, buscar, actualizar y eliminar productos; consultar y gestionar categorías.
Administrador	Realizar las mismas operaciones del Usuario y administrar información desde el panel administrativo de Django.

Límite del sistema: las funciones anteriores pertenecen al sistema de inventario; el panel Django es acceso administrativo. Las asociaciones son simples, sin include ni extend. Los actores y permisos son conceptuales: la autenticación aún no está implementada.

1.4 Diagrama de clases (UML)

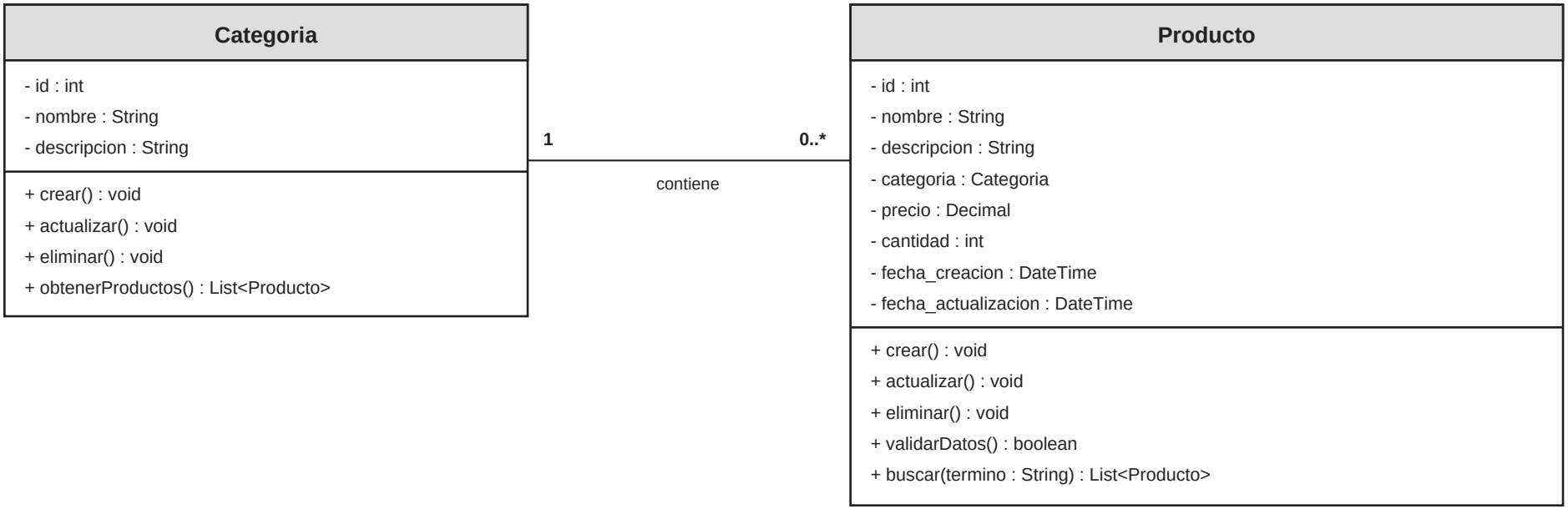


Figura 4. Clases Categoria y Producto con atributos tipados (-), métodos (+) y cardinalidad 1 a 0..*.

Relación: una categoría tiene cero o muchos productos; cada producto pertenece a una sola categoría. El modelo es simple y no requiere herencia ni composición. Los métodos se muestran como stubs propuestos, en coherencia con el apartado 3.3.

1.5 Arquitectura MVT

El repositorio contiene ahora el esqueleto Django; el prototipo visual sigue separado y estático.



Las vistas y plantillas responden, pero todavía no crean, consultan ni modifican datos. Las validaciones y MySQL también están pendientes.

2. Prototipado y navegación

Seis pantallas HTML/CSS estáticas representan rutas principales previstas.

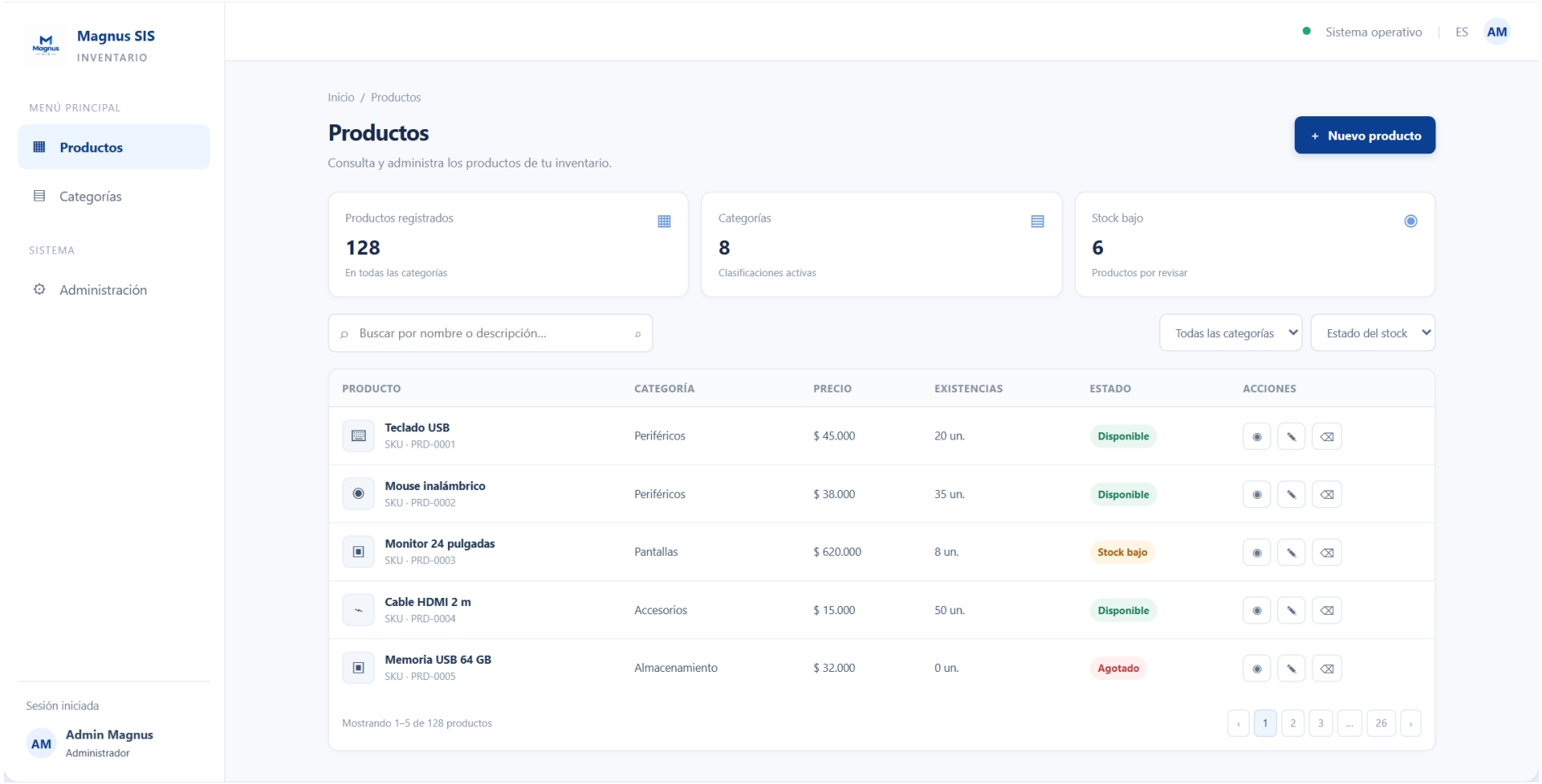
Pantalla	Ruta prevista	Correspondencia
Productos	/	Listado, búsqueda, filtros y stock
Nuevo producto	/productos/nuevo/	Formulario y validación prevista
Detalle	/productos/<id>/	Datos del producto y existencias
Editar / eliminar	/productos/<id>/editar/ · /eliminar/	Formulario y confirmación
Categorías	/categorias/	Listado y acceso visual para crear
Administración	/admin/	Panel Django; fuera del mockup

El prototipo de categorías solo muestra el listado. El formulario para crear o editar categorías queda pendiente. Contadores y ejemplos de productos son ilustrativos.

2.1 Mockup · Inventario de productos

Listado, búsqueda, filtros, existencias y accesos CRUD.

Listado y búsqueda

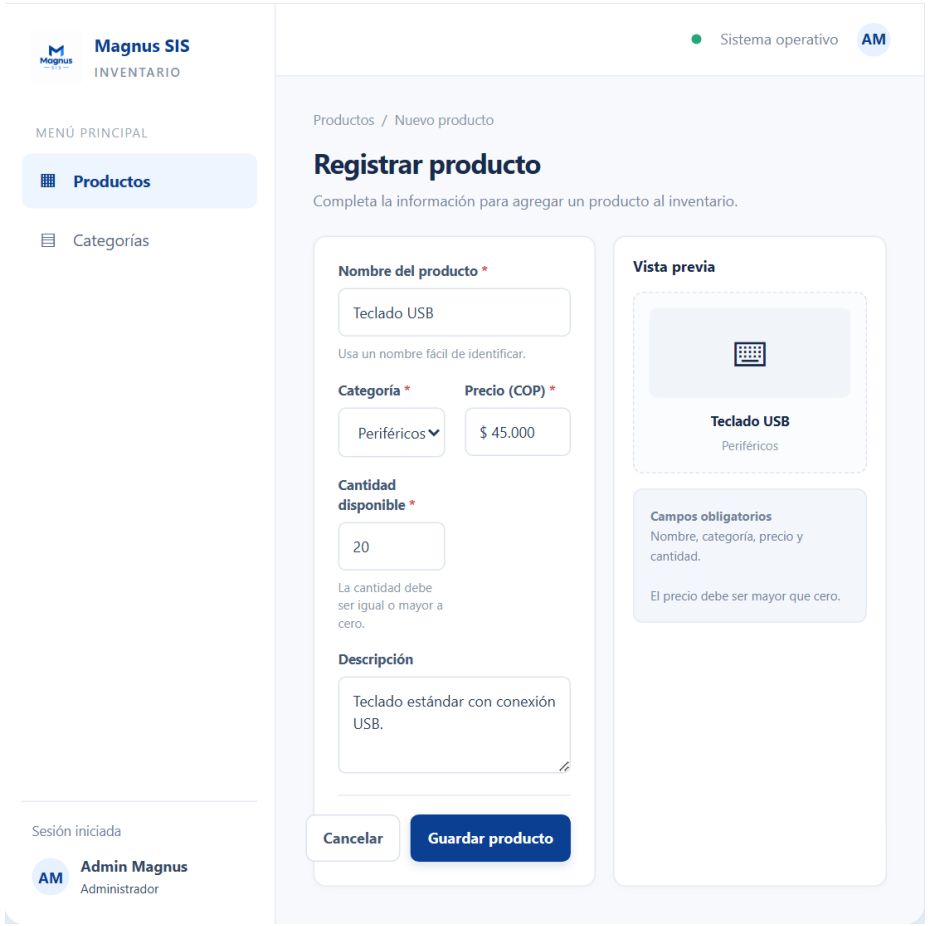


Captura estática. Filas, cifras y estados de stock son datos de ejemplo.

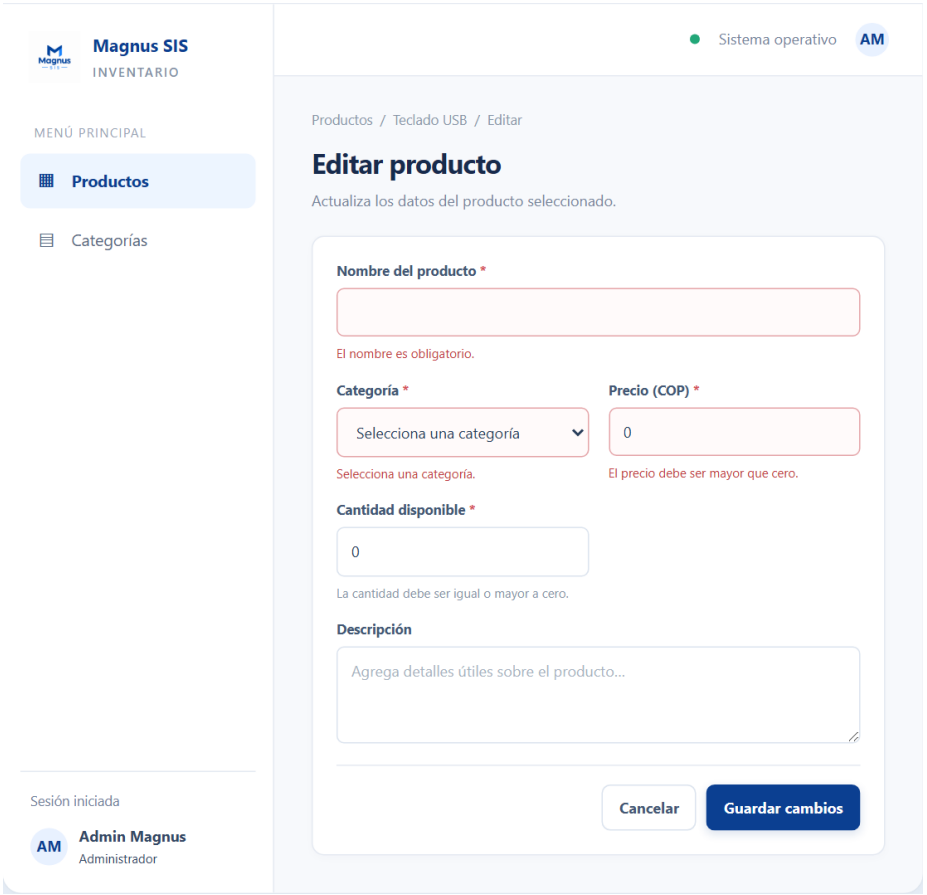
2.2 Mockups · Registro y edición

Formularios y mensajes que corresponden a las validaciones propuestas.

Registrar producto



Editar · mensajes

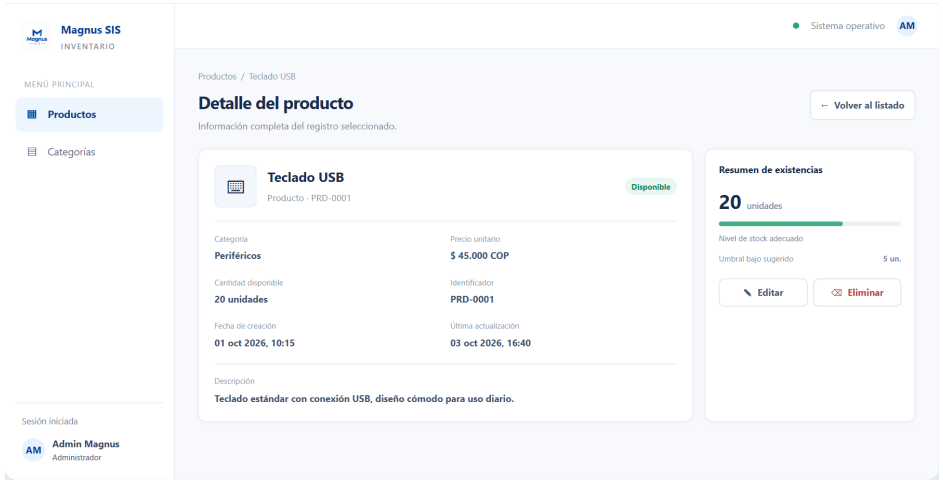


La validación se propone en Django Forms; no está implementada en estas pantallas estáticas.

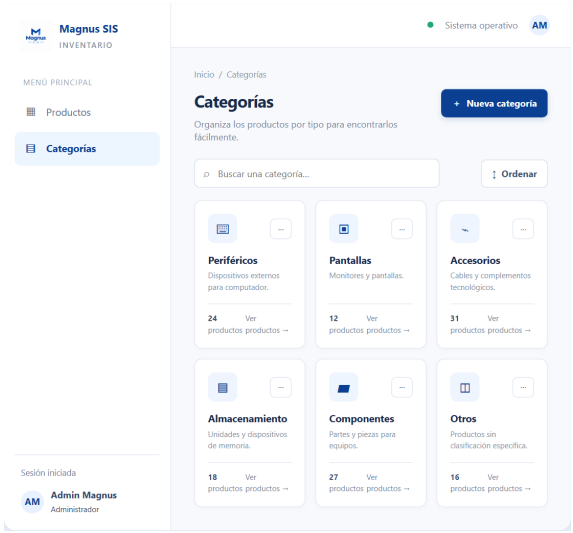
2.3 Mockups · Detalle, categorías y eliminación

Consulta, clasificación y confirmación antes de eliminar.

Detalle



Categorías



Confirmación de eliminación



Los datos son ilustrativos. La pantalla de categorías solo representa el listado; la administración funcional queda pendiente.

3. Esqueleto del código y configuración

Estructura existente en el repositorio; los módulos de inventario están implementados como stubs.

ESTRUCTURA ACTUAL

```
CRUD_MagnusSIS/
■■ .gitignore
■■ README.md
■■ requirements.txt
■■ documento_Sistema.pdf
■■ mockups/
■    ■■ mockups-inventario.html
■    ■■ mockups-inventario.css
■■ magnus_sis/
   ■■ manage.py
   ■■ magnus_sis/{settings,urls,asgi,wsgi}.py
   ■■ productos/
     ■■ models.py · forms.py · views.py
     ■■ urls.py · admin.py · tests.py
     ■■ migrations/0001_initial.py
     ■■ templates/productos/*.html
```

CONFIGURACIÓN Y VERIFICACIÓN

Django está registrado en INSTALLED_APPS y las rutas de productos están incluidas en el proyecto.

SQLite está configurado para desarrollo. La migración inicial crea Categoria y Producto.

Verificado: `python manage.py check` sin errores y dos pruebas de humo aprobadas.

El esqueleto puede ejecutarse; las operaciones de negocio, búsqueda real y validaciones personalizadas se dejan para la siguiente fase.

3.3 Clases y funciones stub · modelos

Implementados en productos/models.py. Los métodos de negocio siguen vacíos o devuelven valores por defecto.

```
class Categoria(models.Model):
nombre = models.CharField(max_length=100)
descripcion = models.TextField(blank=True)
def crear(self) -> None: pass
def actualizar(self) -> None: pass
def eliminar(self) -> None: pass
def obtener_productos(self) -> list: return []

class Producto(models.Model):
nombre = models.CharField(max_length=150)
descripcion = models.TextField(blank=True)
categoria = models.ForeignKey(Categoria, on_delete=models.PROTECT,
related_name="productos")
precio = models.DecimalField(max_digits=10, decimal_places=2)
cantidad = models.PositiveIntegerField(default=0)
fecha_creacion = models.DateTimeField(auto_now_add=True)
fecha_actualizacion = models.DateTimeField(auto_now=True)
def crear(self) -> None: pass
def actualizar(self) -> None: pass
def eliminar(self) -> None: pass
def validar_datos(self) -> bool: return False
@staticmethod
def buscar(termino: str) -> list: return []
```

Los SKU, umbrales y contadores que aparecen en el mockup son ejemplos visuales; no forman parte de estos modelos.

3.3 Formularios y vistas stub

Los formularios y vistas están conectados a templates placeholder; el procesamiento POST está pendiente.

productos/forms.py

```
class ProductoForm(forms.ModelForm):
class Meta:
model = Producto
fields = ["nombre", "descripcion", "categoria", "precio", "cantidad"]
def clean(self) -> dict:
# Pendiente: precio > 0 y validaciones de negocio.
return super().clean()

class CategoriaForm(forms.ModelForm):
class Meta:
model = Categoria
fields = ["nombre", "descripcion"]
```

productos/views.py

```
def lista_productos(request):
return render(request, "productos/lista.html", {})

def buscar_producto(request):
return render(request, "productos/lista.html", {})

def detalle_producto(request, pk):
return render(request, "productos/detalle.html", {})

def crear_producto(request):
return render(request, "productos/form.html", {})

# editar_producto, eliminar_producto y lista_categorias
# siguen el mismo patrón de respuesta placeholder.
```

Los campos obligatorios básicos se derivan del modelo; las reglas de negocio personalizadas aún no se aplican.

3.3 Rutas y pruebas de humo

Rutas incluidas en el proyecto. Las pruebas solo comprueban respuestas HTTP de las plantillas stub.

productos/urls.py

```
app_name = "productos"
urlpatterns = [
path("", views.lista_productos, name="lista"),
path("buscar/", views.buscar_producto, name="buscar"),
path("productos/nuevo/", views.crear_producto, name="crear"),
path("productos/<int:pk>/", views.detalle_producto, name="detalle"),
path("productos/<int:pk>/editar/", views.editar_producto, name="editar"),
path("productos/<int:pk>/eliminar/", views.eliminar_producto,
name="eliminar"),
path("categorias/", views.lista_categorias, name="categorias"),
]
```

productos/tests.py

```
class HumoTests(TestCase):
def test_lista_responde(self):
self.assertEqual(
self.client.get(reverse("productos:lista")).status_code, 200
)
def test_formulario_responde(self):
self.assertEqual(
self.client.get(reverse("productos:crear")).status_code, 200
)
```

3.4 Estructura de datos y configuración

Migración inicial de SQLite creada para los modelos del esqueleto.

MODELOS MIGRADOS

Categoria: id, nombre VARCHAR(100), descripcion TEXT opcional.

Producto: id, nombre VARCHAR(150), descripcion TEXT opcional, categoria_id FK protegida, precio DECIMAL(10,2), cantidad entero ≥ 0, fecha_creacion y fecha_actualizacion.

Una categoría tiene cero o muchos productos; cada producto pertenece a una categoría.

CONFIGURACIÓN

Actual: SQLite en `magnus_sis/settings.py`; Django 6.1.x en `requirements.txt`.

Objetivo posterior: MySQL mediante Django ORM; el conector y las variables de conexión no están configurados todavía.

.gitignore: excluye entornos virtuales, bytecode, bases SQLite y archivos .env.

SKU, stock mínimo, métricas y paginación de las capturas son solo referencias visuales y aún no están modelados ni calculados.

README.md · repositorio y estado

Resumen del proyecto y enlace directo al repositorio.

Sistema Web de Gestión de Inventario · Magnus SIS

Asignatura: Lenguaje de Programación 1 · 2026-2
Integrantes: Kevin Julian Sanchez Perez · Juan Jose Osorio Quitian · Luis Esteban Medina · Brayan Steven Sanchez · Sergio Neira · Hilver Galindo

Estado: Mockup HTML/CSS estático y esqueleto Django con app, modelos, formularios, rutas, vistas, templates, migración y pruebas de humo. Las operaciones CRUD y la lógica de negocio están pendientes.

Repositorio:

https://github.com/kevinsanchezpe-ops/CRUD_MagnusSIS

Verificación local: `python manage.py check` y `python manage.py test productos` completan sin errores.